Chapter 11

Offline Reinforcement Learning

Reading

1. Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems by Levine et al. (2020)
2. Continuous doubly constrained batch reinforcement learning by Fakoor et al. (2021)

So far, we have imagined that we have access to a model of a robot, a simulator for it, or access to the actual robot that allows us to obtain data from the robot. But there are many problems in which we cannot get any of these. For example, when Amazon sells merchandise to its customers, it is quite difficult for them to model or simulate each customer, or even a canonical one. It is possible to do rollouts using actual customer data but that would not be wide because an exploratory policy, by the time it learns, would lead to huge loses. This would also not be desirable for the customers. Amazon also needs reinforcement learning for this problem because there is not many other ways to explore what customers like and do not like. One could take another example from a very different domain. Suppose a hospital is trying to develop a new protocol to handle incoming patients. E.g., a patient comes into ER, there is a fixed set of checks that are performed quickly on them called as "triage". The number and kind of checks are performed directly affects patient care. If there are too many checks, then the patient loses precious time before they are treated. If there are too few, then there is a large bottleneck when these patients are referred to doctors. It is not easy to model or simulate this problem. It is possible to do rollouts to discover a policy but that would not be easy, or wise.

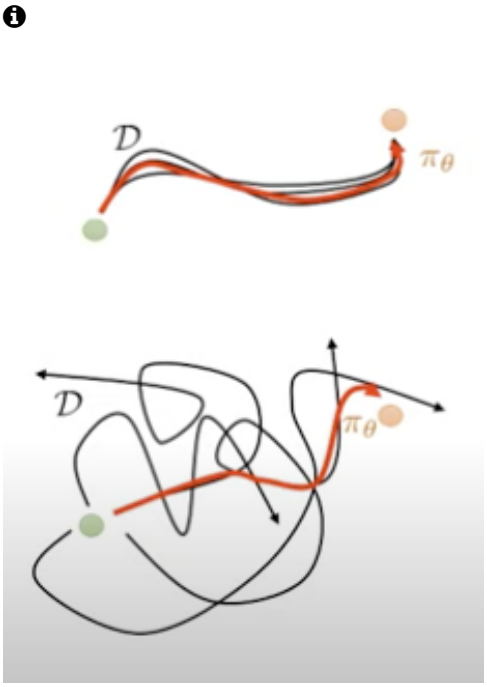
These problems have a few commonalities.

- We do not have models or simulators for such systems *and that is why* we need to use ideas from reinforcement learning to build controllers for them;
- There are both existing systems, i.e., there exists a controller that is currently deployed. This controller may be very complex, e.g., in Amazon's case it is the result of many scientists building this system over a decade (and thereby even if one could look at it, there is no way one would be able to model/simulate it). In the case of the hospital, the current triage policy was likely created by the doctors over experience and refined by the triage nurses by looking at actual cases. Because the system evolved over a long time, a lot of this knowledge is not accessible in a codified form.

Offline reinforcement learning are a suite of techniques that allow us to learn the optimal value function from data that need not be coming from an optimal policy—without drawing any new data from the environment. Note that we do not just want to evaluate an existing policy, we want to learn the optimal value function, or at the very least improve upon the current policy.

There are many other problems of this kind: data is plentiful, just that we cannot get more.

Technically speaking, offline learning is a very clean problem in that we are close to the supervised learning setting (we do not know the true targets). A meaningful theoretical analysis of typical reinforcement learning algorithms is difficult because there are a lot of moving parts in the problem definition: exploratory controllers, the fact that we are adding correlated samples into our dataset as we draw more trajectories, function approximation properties of the neural network that does not allow Bellman iteration to remain a contraction etc. Some of these hurdles are absent in the analysis of offline learning.



A good intuition for offline reinforcement learning is given by this picture. In imitation learning, or behavior cloning, we simply want to learn a policy that mimics the expert (or recorded data). In offline reinforcement learning, we can "stitch together" multiple sub-optimal policies in different parts of the domain.

11.1 Why is offline reinforcement learning difficult?

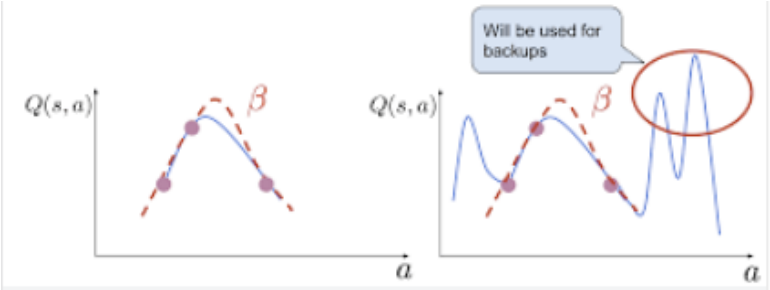
Suppose that we wanted to use behavior cloning for this problem. After all, we could build a state vector x and do behavior cloning with a neural network to learn a policy $u_\theta(x)$ from the existing dataset $D = \left\{(x_t^i, u_t^i)_{t=0,\ldots,T}\right\}_{i=1}^n$. In principle, we can also also learn the value function corresponding to this policy, $q_\varphi^\theta(x, u)$ where u is the control input. Note that this is not the optimal value function. We will call this the behavior cloning solution to offline reinforcement learning.

We know further that value iteration converges (for tabular MDPs) from any initial condition. In lieu of the actual model of the Markov Decision Process, we can imagine using the entire dataset D to calculate

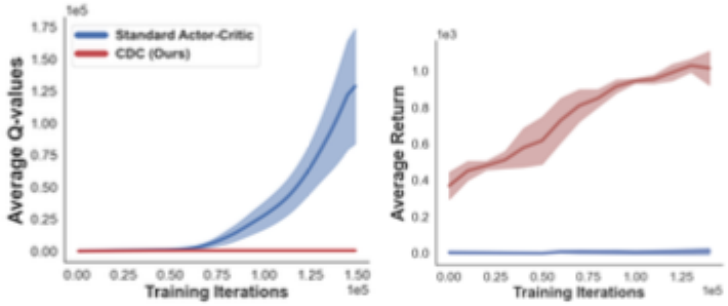
Bellman updates for a value function parameterized by a neural network:

$$\min_{\varphi} \sum_{i,t} \left(q_{\varphi}^{\theta\,(k+1)}(x_t^i, u_t^i) - r(x_t^i, u) - \gamma \max_{u' \in U} q_{\varphi}^{\theta\,(k)}(x_{t+1}^i, u') \right)^2 .$$

This approach is unlikely to work very well. Observe the following figure.



We do not really know whether the initial value function q_{φ}^{θ} assigns large returns to controls that are outside of the ones in the dataset. If it does, then these controls will be chosen in the maximization step while calculating the target. If there are states where the value function over different controls looks like this picture, then their targets will cause the value at all other states to grow unbounded during training. This is exactly what happens in practice. For example,



In offline reinforcement learning, this phenomenon is often called the "extrapolation error". It arises because we do not know a natural way to force the network to avoid predicting large values for controls that are not a part of the training dataset. It is instructive to ask why off-policy or on-policy reinforcement learning does not suffer from extrapolation error. Both of these algorithms explicitly draw more data from the simulator. If the value function were to over-estimate the value of certain control actions at a state, then the controller would take those controls during exploration and discover that the value was in fact an over-estimate. Methods that draw more data have this natural self-correcting behavior that we cannot get in offline learning.

There is a second problem associated with computing the maximum in value iteration. For problems with continuous controls, we do not know of computationally effective ways to compute the maximum $\max_{u \in U}$.

ℹ We have discussed how Bellman updates are a set of consistent constraints on the optimal value function. This entails that if we over-estimate the value at a given state, then the estimated return to come (i.e., the value) of *all* other states becomes incorrect.

Typical implementations of offline learning fit a controller that maximizes the value function

$$\theta = \text{argmax}\, \frac{1}{nT} \sum_{i,t} q^{\theta}_{\varphi}(x^i_t, u_\theta(x))$$

to make it easy to calculate this maximum. But this forces us to use another network (and it is not a given that the parameterized controller can calculate the correct maximum).

ℹ One can use a linear value function, e.g., $q^{\theta}_{\varphi}(x,u) = \langle [1,x,u], \varphi[1,x,u]\rangle$ or any other set of basis functions. But this is not enough to resolve the issue in theory. Bellman updates are not a contraction when the TD-error is projected onto a set of bases. In practice, this approach works reasonably well.

11.2 Regularized Bellman iteration

There are two broad class of techniques that are believed to give reasonable results for offline learning. These are both quite new and ad hoc and effective offline learning is essentially an open problem today. To wit, current offline learning methods not only fail to learn optimal value functions or policies from sub-optimal data, but they often do any better than behavior cloning.

11.2.1 Changing the fixed point of the Bellman iteration to be more conservative

Since extrapolation error is fundamentally caused by the value function taking large values, it is reasonable strategy to modify the Bellman updates to regularize the value function in some way. A basic version would look like a coupled system of updates

$$\begin{aligned}\varphi^* = \underset{\varphi}{\text{argmin}}\, \frac{1}{nT} \sum_{i,t} \left(q^{\theta\,(k+1)}_{\varphi}(x^i_t, u^i_t) - r(x^i_t, u) - \gamma q^{\theta\,(k)}_{\varphi}(x^i_{t+1}, u_\theta(x^i_{t+1}))\right)^2 \\ - \lambda\, \Omega(q^{\theta}_p) \\ \theta^* = \underset{\theta}{\text{argmax}}\, \frac{1}{nT} \sum_{i,t} q^{\theta}_{\varphi}(x^i_t, u_\theta(x^i_t)).\end{aligned} \tag{11.1}$$

We will use the regularizer

$$\Omega = \frac{1}{nT} \sum_{i,t} \left(q^{\theta}_{\varphi}(x^i_t, u_\theta(x^i_t)) - q^{\theta}_{\varphi}(x^i_t, u^i_t)\right)^2_+$$

The notation $(\cdot)_+$ denotes rectification, i.e., $(x)_+ = x$ if $x > 0$ and zero otherwise. Notice that second term in the objective for fitting the value function forces the value of the control $u_\theta(x^i_t)$ to be smaller than the value of the control u^i_t taken at the same state in the dataset. This is a conservative choice. While it does not fix the issue of extrapolation error, it forces the value network to predict smaller values and prevents it from blowing up.

A second, similar, strategy looks like

$$\Omega = \frac{1}{nT} \sum_{i,t} \max_{u' \sim u_\theta(x_t^i)} \left(q_\varphi^\theta(x_t^i, u') - q_\varphi^\theta(x_t^i, u_t^i) \right)_+^2$$

where the maximum of actions is computed over a large number of samples. A yet another strategy looks like

$$\Omega = \frac{1}{nT} \sum_{i,t} \log \int_u \exp\left(q_\varphi^\theta(x_t^i, u) \right) \mathrm{d}u \,.$$

These strategies have been found to be somewhat useful in the sense that they prevent the value function from taking large values. But it is also clear that the solution of the problems is not the optimal value function.

Practically speaking, it is reasonable to expect that even if we do not find the optimal value function, if we can obtain a better policy than the existing policy that is running on the system, then offline reinforcement learning is a viable approach.

11.2.2 Estimating the uncertainty of the value function

It is reasonable to ask the question whether initializing the value function

Bibliography

Fakoor, R., Mueller, J. W., Asadi, K., Chaudhari, P., and Smola, A. J. (2021). Continuous doubly constrained batch reinforcement learning. *Advances in Neural Information Processing Systems*, 34:11260–11273. 1

Levine, S., Kumar, A., Tucker, G., and Fu, J. (2020). Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*. 1